M1：資料思維與 Notebook 工作流

課程定位： 第二幕 — 建立資料思維底盤，掌握分析師的基礎工作環境 時數： 3 小時 前置模組： M0 開場全景 後續模組： M2 Python 分析基礎

模組學習目標

完成本模組後，學生能夠：

1. 說出「資料分析工作」的真實樣貌，區分清理、探索、推論三個核心動作
2. 識別結構化資料的五個基本元素：列、欄、綱要、型態、鍵
3. 判斷一份資料是否「乾淨」，說出四類常見污染並提出初步處理策略
4. 解釋 Excel 思維與 Python 思維的根本差異，以及選擇時機
5. 說明 Jupyter Notebook 的四合一設計哲學，並在實際環境中操作
6. 辨識常見資料格式（CSV、JSON、SQL 表格、API 回應）的結構特徵
7. 套用資料品質評估框架，對一份新資料進行系統性的健康檢查
8. 使用 Notebook 把分析過程組織成有推論脈絡的敘述，而不只是程式碼輸出

關鍵概念清單

完成模組後，以下每個概念都應能一句話解釋：

- [ ] 資料分析的三個核心動作：清理 / 探索 / 推論
- [ ] 結構化資料：列（觀測值）與欄（變數）
- [ ] 綱要（schema）：欄名 + 型態的完整描述
- [ ] 鍵（key）：唯一識別一筆記錄的欄位
- [ ] 缺失值（missing value）：`NaN`、空字串、假值的差異

- [ ] 重複值（duplicate）：完全重複 vs. 邏輯重複
- [ ] 離群值（outlier）：異常的數值，不一定是錯誤
- [ ] 型態錯誤（type error）：數字被存成字串的情況
- [ ] 資料品質評估框架：完整性 / 一致性 / 準確性 / 時效性
- [ ] CSV：純文字的表格格式，欄位以逗號分隔
- [ ] JSON：鍵值結構，巢狀資料的標準格式
- [ ] SQL 表格：關聯式資料庫的儲存單位，有嚴格型態定義
- [ ] API 回應：動態取得的資料，通常為 JSON 結構
- [ ] Excel 思維：手動操作，難以重現
- [ ] Python 思維：可重現工作流，腳本即文件
- [ ] Jupyter Notebook：文字 + 程式 + 圖表 + 推論四合一
- [ ] 資料分析生命週期：提問 -> 取得 -> 清理 -> 探索 -> 推論 -> 呈現

投影片大綱

#	投影片標題	核心訊息	預估時間
S01	你以為的資料分析 vs. 實際在做什麼	資料分析不是「畫圖表」，是清理 + 探索 + 推論的完整循環	12 min
S02	資料分析生命週期：六個階段	從提問到呈現，每個階段都有明確任務和輸出	12 min
S03	結構化資料的五個元素	列、欄、綱要、型態、鍵，看懂這五個才能開始工作	15 min
S04	什麼叫做「乾淨的資料」	四種污染類型：缺失、重複、離群、型態錯誤，各有處理邏輯	15 min
S05	資料品質評估框架	完整性、一致性、準確性、時效性，系統性而不是憑感覺	12 min

S06	認識常見資料格式	CSV、JSON、SQL 表格、API 回應，各自的結構特徵與使用場景	15 min
S07	Excel 思維 vs. Python 思維	Excel 是手動操作；Python 是可重現的流程腳本	12 min
S08	為什麼 Jupyter Notebook 成為標準	文字 + 程式 + 圖表 + 推論四合一，分析過程可被閱讀與重現	12 min
S09	Notebook 工作流實戰技巧	cell 管理、Markdown 撰寫、執行順序、輸出管理的最佳實踐	15 min
S10	工作坊 A：初探一份真實資料	用 Notebook 的標準工作流，對一份新資料進行系統性探索	30 min
S11	工作坊 B：資料品質健康檢查	套用品質評估框架，找出資料問題並記錄在 Notebook 中	30 min
S12	模組總結：分析師的思維工具箱	資料思維 + Notebook 工作流是所有後續分析技術的地基	10 min

總計：約 190 分鐘（含工作坊與 Q&A）

詳細投影片內容

S01 — 你以為的資料分析 vs. 實際在做什麼

核心訊息：資料分析不是「畫圖表」，是清理 + 探索 + 推論的完整循環。圖表只是最後的呈現，不是工作的主體。

講師講解要點： - 多數初學者以為資料分析 = 做 dashboard / 做折線圖。這是誤解。圖表是最末一步，前面有大量清理和判斷工作。 - 資料分析師的實際時間分配：業界調查顯示約 60-80% 時間花在資料清理，而非建模或視覺化。 - 三個核心動作定義： - 清理：讓資料可信。沒有這一步，後續所有結果都可能是錯的。 - 探索：找出值得問的問題。資料裡有什麼樣的分布？有沒有異常？有沒有意外的相關性？ - 推論：用資料支撐決策。這裡的「推論」不必然是統計推論，也包括業務判斷：「這個指標下降，是因為活動結 - 強調「推論」需要業務背景，純技術人員容易在這一步出問題：數字算出來了，但解讀錯了。 - 這門課從 M1 到 M6 都在訓練這條鏈，從資料思維和 Python 基礎出發，不是從圖表出發。

視覺建議： 兩欄對比圖。左欄「你以為的流程」：資料 → 圖表 → 結論（三步，箭頭乾淨）；右欄「實際流程」：原始資料 → 清理（缺失 / 重複 / 型態）→ 探索（分布 / 相關 / 異常）→ 推論（假設 / 驗證 / 決策）→ 呈現。用多個循環箭頭暗示這是迭代而非線性。右欄中特別標注「60-80% 時間在這裡」指向清理階段。

課堂互動： 問學員：「你們有沒有用 Excel 做過資料分析？最花時間的步驟是什麼？」讓 2-3 位學員回答，帶出清理工作的普遍性。

過渡語：
「要能做好這三件事，我們需要一個系統性的框架，不能每次拿到資料都從零開始想。這個框架叫做資料分析生

S02 — 資料分析生命週期：六個階段

核心訊息：
一個完整的資料分析任務包含六個階段：提問、取得、清理、探索、推論、呈現。每個階段都有明確的輸入、任

講師講解要點：

第一階段：提問（Define the Question） - 資料分析最重要的一步，也最常被跳過。 - 好問題的特徵：可以用資料回答、有業務意義、結果能影響決策。 - 壞問題的例子：「分析銷售資料」（太廣泛，無法知道什麼時候算「分析完了」）。 - 好問題的例子：「上個月北部地區的退貨率是否高於全國平均，以及退貨最集中的商品類別是哪三個。」

第二階段：取得（Acquire Data） - 來源多樣：CSV 文件、資料庫查詢、API 呼叫、網頁爬蟲、人工收集。 - 取得時就要記錄：資料來源、時間範圍、已知限制。 - 常見陷阱：取得的資料不是你真正需要的（欄位不對、時間範圍不對、抽樣偏差）。

第三階段：清理（Clean） - 處理缺失、重複、離群、型態錯誤（S04 詳細說明）。 - 清理決策要記錄：為什麼刪掉這些行？為什麼用均值填補？這些決策會影響最終結論。

第四階段：探索（Explore） - 又稱 EDA（Exploratory Data Analysis）。 - 看分布（每個欄位的數值範圍是什麼）、看相關（哪些欄位之間有關係）、看異常（有沒有不符預期的模式）。 - 探索的目的是找出值得深入的方向，不是給出結論。

第五階段：推論（Analyze / Infer） - 用統計方法或業務邏輯，把探索發現轉化為可支撐決策的結論。 - 注意：「相關性不等於因果性」是這個階段最重要的警惕。

第六階段：呈現（Communicate） - 面向不同受眾，呈現方式不同：給技術人員看 Notebook，給管理層看簡報，給系統用 API。 - 好的呈現不是「圖表漂亮」，而是「讓決策者能正確理解並行動」。

視覺建議：
六個色塊排成一排的生命週期流程圖，每個色塊包含：階段名稱（中英文）、輸入、關鍵任務（一行）、輸出。↔ 探索之間特別加粗，代表經常來回）。

過渡語：「生命週期的第一件實際工作，是讀懂資料本身的結構。一份資料，不管是 CSV 還是資料庫，它背後都有五個基本元素。」

S03 — 結構化資料的五個元素

核心訊息：
每一份結構化資料都可以用列（row）、欄（column）、綱要（schema）、型態（type）、鍵（key）來完整描

講師講解要點： - 以一張真實感的訂單資料表為例（訂單 ID / 客戶名稱 / 金額 / 日期 / 狀態），帶出五個元素。

列（row）= 一筆觀測值 - 代表「一件事發生了」。一列訂單 = 一筆交易發生。 - 列的數量 = 這份資料記錄了多少件事。 - 在 pandas 中，`df.shape[0]` 就是列數。

欄（column）= 一個變數 - 描述這件事的一個面向。金額是一個欄，日期是另一個欄。 - 欄的數量 = 我們對每件事記錄了多少個面向。 - 在 pandas 中，`df.columns` 列出所有欄名。

綱要（schema）= 欄名 + 型態的完整定義 - 是資料的「藍圖」。換一份新資料，第一件事是看 schema。 - 例子：`order_id: int, customer_name: str, amount: float, order_date: datetime, status: str`。 - 在 pandas 中，`df.dtypes` 顯示每欄的型態（近似 schema）。

型態（type）= 每個欄的資料類型 - 整數（int）、浮點數（float）、字串（str）、日期（datetime）、布林（bool）。 - 型態決定能做哪些運算：數字型可以加減乘除，字串型可以搜尋替換，日期型可以計算時間差。 - 型態不對是資料問題的重大來源（S04 詳細說明）。

鍵（key）= 唯一識別一筆記錄的欄位 - 通常是 ID 類欄位，保證每一列都是獨立的、不重複的記錄。 - 沒有鍵或鍵不唯一：合併兩份表格時會出現笛卡爾積爆炸（一列變成多列）。 - 自然鍵 vs. 代理鍵：`■■ ID` 是代理鍵（系統生成），`■■■■■` 是自然鍵（現實世界存在）。

實際操作示範：

```
import pandas as pd

df = pd.read_csv("orders.csv")

# ■■■■■■tuple; ■■, ■■■
print(df.shape)

# ■ schema■■■ + ■■■
print(df.dtypes)

# ■■■■
df.head()

# ■■■■■■■
print(df["order_id"].nunique() == len(df))  # True ■■ order_id ■■■■■
```

視覺建議：
一張標記過的資料表格截圖，用不同顏色箭頭標出：橫向掃一列（標「一筆觀測值」）、縱向掃一欄（標「一個 ID 欄標「key」、欄標題下方標型態。旁邊對應 pandas 指令。

過渡語：
「看得懂結構，下一步是判斷資料有沒有問題。大部分初學者拿到資料就開始分析，但乾淨的資料才是分析的前

S04 — 什麼叫做「乾淨的資料」

核心訊息：
污染資料有四種形式：缺失值、重複值、離群值、型態錯誤。識別這四類是清理工作的第一步，每一類都有不同

講師講解要點：

缺失值（missing value） - 欄位沒有值。在 Python/pandas 中通常是 `NaN`（Not a Number）。 - 重要區分：空字串 `""` 和假值 `0` 在有些系統中被用來表示「沒有資料」，但它們不是 `NaN`，處理方式不同。 - 三種處理策略： - 刪除：缺失比例極高（> 50%），或缺失本身就代表「這筆記錄無效」。 - 填補：用均值、中位數、眾數，或用業務規則填補（缺失的退貨率可能代表 0）。 - 保留：缺失本身有意義，代表「沒有發生」，不應該刪掉。

```
[illegible]
df.isna().sum()
[illegible]
[illegible]
df.isna().mean() * 100
```

重複值（duplicate） - 完全重複：整列所有欄位都相同，通常是匯入錯誤（同一批資料被匯入兩次）。 - 邏輯重複：同一個客戶用不同拼法出現兩次（`Alice Wang` vs. `alice wang`），這更難自動抓，需要人工或模糊比對。

```
[illegible]
print(df.duplicated().sum())
[illegible]
[illegible]
df_clean = df.drop_duplicates()
```

離群值（outlier） - 極端的數值。注意：離群值不一定是錯誤。 - 判斷邏輯：一位真實的頂級客戶可能消費是平均值的 100 倍，這是真實的業務現象，不是錯誤。 - 需要先判斷「是資料輸入問題」還是「是真實的業務現象」，再決定是否處理。 - 常見判斷方式：統計描述（`.describe()`）+ 分布圖（M4 會教）。

型態錯誤（type error） - 金額欄位被存成字串 `"$1,200"`、日期被存成 `"2024/01/15"` 字串而非 datetime。 - Python 計算時會報錯，或更糟，默默產生錯誤結果（字串相加 `"1" + "2" = "12"` 不是 3）。 - 清理時要明確轉換型態，並處理轉換失敗的情況。

```
[illegible]
df["amount"] = (
    df["amount"]
    .astype(str)
    .str.replace(r"[$,]", "", regex=True)
    .astype(float)
)
```

清理的目的：

不是讓資料「好看」，是讓分析「可信」。沒有清理就直接分析，結論可能是完全錯誤的。

視覺建議：

四格圖，每格對應一種污染類型，用同一張訂單表格呈現，高亮出問題儲存格：缺失值用灰底、重複列用黃底標

過渡語：

「面對一份新資料，需要有系統性的檢查方法，不能只憑直覺。下一張投影片介紹一個可以反覆套用的評估框架

S05 — 資料品質評估框架

核心訊息：

系統性評估資料品質，用完整性、一致性、準確性、時效性四個維度，取代憑感覺的「看一看」，讓每次資料檢

講師講解要點：

完整性（Completeness） - 問題：資料應該有的記錄，都在嗎？ - 檢查方向： - 欄位層面：每欄的缺失比例是多少？哪些欄位的缺失是不可接受的？ - 記錄層面：預期應該有 30 天的資料，結果只有 28 天，哪兩天不見了？ - 覆蓋層面：這份資料只有「成功訂單」，「取消訂單」呢？

```
[redacted]
def check_completeness(df):
    missing_pct = df.isna().mean() * 100
    print("=== [redacted] ===")
    for col, pct in missing_pct.items():
        status = "OK" if pct == 0 else f"WARNING: {pct:.1f}% missing"
        print(f"  {col}: {status}")
```

一致性（Consistency） - 問題：同一個事實，在不同地方的記錄一致嗎？ - 常見問題： - 同一個人名在不同資料集有不同拼法（`alice` / `Alice` / `ALICE`）。 - 同一個金額在不同欄位記錄不一致（`amount = 1200` 但 `items × price = 1300`）。 - 日期格式不統一（`2024-01-15` vs. `15/01/2024`）。

準確性（Accuracy） - 問題：記錄的值，符合現實嗎？ - 這是最難自動化的一個維度，通常需要業務知識輔助。 - 常見問題：年齡欄位出現 200 歲、出生日期晚於購買日期、退貨率 > 100%。

時效性（Timeliness） - 問題：資料是否足夠新，能用來回答當前問題？ - 舉例：用三年前的顧客行為資料來預測現在的購買意願，可能已過時。 - 檢查方向：資料的最新更新時間、是否有資料延遲的已知問題。

套用框架的標準輸出：
每次拿到新資料，應該產出一份「資料品質摘要」，包含：四個維度的評估結果、已知問題列表、建議的清理步 Notebook 的開頭。

視覺建議： 四個維度的評分卡格式，每個維度包含：維度名稱 + 核心問題 + 檢查方法 + 常見問題例子。底部加上「輸出：資料品質摘要」方框。

過渡語： 「框架告訴你要看什麼，但資料本身有不同的長相。一份 CSV 和一份 JSON 和一個資料庫表格，結構上有什麼異同？」

S06 — 認識常見資料格式

核心訊息： 資料分析師會遇到四種主要資料格式：CSV、JSON、SQL 表格、API 回應。每種格式有其設計用途和結構特徵，讀懂格式是取得資料之後的第一步。

講師講解要點：

CSV（Comma-Separated Values） - 最簡單的表格格式。純文字，每行是一列記錄，欄位用逗號（或 Tab）分隔。 - 優點：任何工具都能讀，人可以直接閱讀。 - 缺點：沒有型態資訊（所有欄位都是字串），沒有架構定義，大檔案效能差。 - 使用場景：資料匯出 / 匯入、小規模資料交換、Excel 的通用格式。

```
import pandas as pd

df = pd.read_csv("data.csv")
# 指定型態
df = pd.read_csv("data.csv", dtype={"amount": float, "order_id": str})
```

JSON（JavaScript Object Notation） - 鍵值對結構，支援巢狀（nested）資料。 - 資料範例：

```
{
  "order_id": "ORD-001",
  "customer": {
    "name": "Alice Wang",
```

```
  "tier": "premium"
},
"items": [
  {"product": "A", "qty": 2, "price": 150},
  {"product": "B", "qty": 1, "price": 300}
]
}
```

- 優點：支援複雜的巢狀結構，是 API 回應的標準格式。
- 挑戰：巢狀結構需要「展平（flatten）」才能放進 pandas 的二維表格。

```
import json
import pandas as pd

with open("orders.json") as f:
    data = json.load(f)

[illegible]
df = pd.json_normalize(data)
```

SQL 表格 - 存在關聯式資料庫中（MySQL、PostgreSQL、SQLite 等）。 - 特點：有嚴格的型態定義（建立時就指定），有索引和外鍵關係，支援大規模資料。 - 分析師通常用 SQL 查詢先篩選資料，再用 pandas 做細部分析。

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("sales.db")
# [illegible] SQL [illegible] DataFrame
df = pd.read_sql_query("SELECT * FROM orders WHERE status = 'completed'", conn)
conn.close()
```

API 回應 - 透過 HTTP 請求動態取得的資料，通常為 JSON 格式。 - 與靜態檔案的差異：資料是即時的，但有速率限制（rate limit），可能需要分頁（pagination）。 - 取得 API 資料後，通常先轉成 pandas DataFrame 再分析。

```
import requests
import pandas as pd

response = requests.get("https://api.example.com/sales?date=2024-01-01")
data = response.json()  # [illegible] list of dicts
df = pd.DataFrame(data)
```

格式選擇指南（簡表）：

場景	建議格式
小資料，需要人可讀	CSV
複雜巢狀結構、API 交換	JSON
大規模資料、需要查詢	SQL
即時資料、第三方服務	API（JSON）

視覺建議：
四個並排方框，每個框代表一種格式，包含：格式名稱、簡短範例（程式碼片段）、優點、限制、讀入 pandas 的方式。

過渡語：「看懂了資料的格式和結構，接下來要談工具的選擇。同樣的任務，Excel 和 Python 的做法有什麼根本差異？」

S07 — Excel 思維 vs. Python 思維

核心訊息：Excel 是「手動操作」，Python 是「可重現的工作流」。前者快速直觀，後者需要多寫一點，但腳本本身就是文件，也是自動化的起點。選擇哪

講師講解要點：

Excel 的核心問題不是「不好用」 - Excel 的問題是「操作不留痕跡」。 - 你按了幾十個步驟刪除了某些行、改了某些格式、套用了某個公式，這些動作沒有被記錄下來。 - 下週來一批新資料，你必須重頭做一遍，而且你可能記不清自己上次做了什麼。 - 更嚴重的問題：別人無法驗證你做了什麼，你的結論的可信度無法被獨立審計。

Python 腳本的每一行都是一個可讀的決策 - 「把金額欄轉成 float」、「刪掉缺失值超過 50% 的列」、「日期格式統一成 YYYY-MM-DD」。 - 這些決策是透明的、可讀的、可以被審計的，也可以被修改和改進。 - 可重現性（reproducibility）：相同的輸入 + 相同的腳本 = 相同的輸出，永遠如此。

什麼情況用哪個：

情境	建議工具
快速看一眼資料，一次性	Excel
需要重複執行的分析流程	Python
分析結果需要被別人驗證	Python
資料超過 100 萬行	Python（Excel 有行數限制）
複雜的自訂計算邏輯	Python
和他人協作、版本控制	Python（Git 可追蹤）

不是說「Excel 沒用」 - 工具選對了，才是好的工程判斷。 - 一份 100 列的一次性對照表，Excel 比 Python 快。 - 一個每月執行的業績報表，Python 比 Excel 安全。

視覺建議： 左右對照時間軸圖。左側「Excel 工作流」：下載 CSV → 打開 Excel → 手動篩選 → 手動刪行 → 用公式轉換 → 複製到 PowerPoint，標注「下次要全部重來」。右側「Python 工作流」：下載 CSV → 執行 `clean.py` → 執行 `analyze.py` → 輸出報告，標注「下次只要換一行檔名」。

課堂互動： 請學員分享一個他們曾經用 Excel 做過、後來很希望是用程式碼做的分析任務。

過渡語： 「Python 思維的最佳實踐環境，是 Jupyter Notebook。它不只是一個 IDE，它改變了資料分析工作的呈現方式。」

S08 — 為什麼 Jupyter Notebook 成為標準

核心訊息： Jupyter Notebook 把文字說明、程式碼、執行結果、圖表全部放在同一個文件裡，讓分析過程本身可以被閱讀、被重現、被分享。

講師講解要點：

傳統分析流程的問題 - 程式在 `.py` 檔、說明在 Word、圖表在 PowerPoint，三個地方分開，難以維護。 - 更新一個地方其他兩個不會自動更新，容易出現不一致。 - 三個月後回頭看，很難重建「當時為什麼這樣做」的推論脈絡。

Notebook 的設計哲學：可執行的論文 - 每個 cell 可以是 Markdown 說明，也可以是可執行的 Python 程式。 - 執行結果（數字輸出、圖表）直接顯示在程式碼下方，不需要截圖或複製貼上。 - 一個 `.ipynb` 文件包含： - 文字（Markdown cell）：解釋為什麼這樣做，業務背景，分析假設 - 程式（Code cell）：實際的操作步驟 - 數字輸出：中間計算結果，方便隨時驗證 - 圖表：視覺呈現，直接嵌入文件

「資料故事」vs. 「資料報告」 - 報告：程式 + 圖表，讀者需要自己理解你在做什麼。 - 故事：推論脈絡，讀者能從頭到尾跟著你的思考走，理解每一步的為什麼。 - 一個好的 Notebook 是「可被任何人閱讀的分析過程」，不只是程式的執行記錄。

Notebook 在實際工作中的定位 - 探索分析（EDA）和溝通：Notebook 是最佳工具。 - 生產環境的自動化執行：通常用 `.py` 腳本，更容易測試和部署。 - 這兩個工具不互斥，Notebook 通常是 `.py` 腳本的前身（先探索，再固化為腳本）。

視覺建議： 一個真實 Jupyter Notebook 截圖，用標記框分別圈出：Markdown 說明 cell（標「文字說明：為什麼這樣做」）、程式碼 cell（標「可執行程式：實際操作」）、數字輸出（標「中間結果：隨時驗證」）、matplotlib 圖表輸出（標「視覺呈現：直接嵌入」）。

過渡語： 「知道了 Notebook 為什麼好，接下來要學怎麼用得好。有一些工作流技巧，讓你的 Notebook 從『能跑』變成『能讀』。」

S09 — Notebook 工作流實戰技巧

核心訊息： Notebook 容易用，但容易用得亂。掌握 cell 管理、Markdown 撰寫、執行順序控制、輸出管理這四個工作流習慣，讓你的 Notebook 從「能跑」變成「能讀」。

講師講解要點：

技巧一：Cell 管理原則 - 每個 code cell 做一件事，不要把十步操作塞進一個 cell。 - 長函式定義放在一個 cell，呼叫函式放在另一個 cell，方便分開重跑。 - 常用快捷鍵（Jupyter Lab / Classic Notebook 通用）： - `Shift + Enter`：執行當前 cell，移到下一個 - `B`（命令模式）：在下方新增 cell - `D, D`（命令模式）：刪除當前 cell - `M`（命令模式）：切換為

Markdown cell - `Y`（命令模式）：切換為 Code cell - `Esc`：進入命令模式

技巧二：Markdown 的標準段落結構 每個分析段落應該有： 1. 目的 cell（Markdown）：這一段要回答什麼問題？ 2. 程式 cell（Code）：執行分析操作。 3. 觀察 cell（Markdown）：從輸出看到了什麼？

```
# [illegible]

[illegible]

[illegible] 0–15% [illegible] 3% [illegible] 50%[illegible]
[illegible]
```

技巧三：執行順序的陷阱 - Notebook 的 cell 可以任意順序執行，這是彈性，也是風險。 - 常見問題：後面的 cell 修改了一個變數，前面的 cell 再執行時行為不同。 - 好習慣：完成分析後，「Restart & Run All」一次，確保從頭到尾順序執行的結果是正確的。 - 左邊的 `[■■]` 顯示執行順序，`[*]` 代表正在執行。

技巧四：輸出管理 - 不是所有中間輸出都需要顯示。用 `;` 或 `_ =` 可以抑制不需要的輸出。 - 對於重要的中間結果，加 Markdown cell 解釋它的意義。 - 提交前清除所有輸出（Kernel → Restart & Clear Output），避免提交到 Git 的 `.ipynb` 包含大量輸出。

```
[illegible]
df.head();  # [illegible]

[illegible]
print(f"[illegible] {len(df_clean)} [illegible] {len(df) - len(df_clean)} [illegible]")
```

技巧五：Notebook 的標準開頭結構

```
# [illegible]

[illegible] Notebook [illegible]
**[illegible]** [illegible] / API [illegible] / [illegible]

[illegible]

**[illegible]** [illegible]
```

視覺建議： 左側一個「好的 Notebook 結構」示範圖，右側標注五個技巧的對應位置。底部加一行：「評判一個 Notebook 的標準：去掉所有輸出後，你還能讀懂它嗎？」

過渡語： 「技巧說完了，現在要動手做。工作坊 A 讓你從頭到尾對一份資料做系統性的探索。」

S10 — 工作坊 A：初探一份真實資料

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook（已預先安裝） 資料： `sales_sample.csv`（課程頁面提供）

目標：使用 Notebook 的標準工作流，套用資料分析生命週期的前三個階段（提問、取得、清理），對一份新資料進行系統性探索，並 Notebook 中記錄每個步驟的觀察。

工作坊 A 任務說明

資料背景：一份假設的電商月銷售資料，包含：訂單 ID、日期、商品類別、銷售金額、訂單數量、退貨率、客戶地區。

Step 0：建立 Notebook 標準開頭（3 分鐘）

新建一個 Notebook，第一個 cell 用 Markdown 寫：

```
# 工作坊 A：[illegible]
> [illegible]
> **資料：** sales_sample.csv
> **目標：** [illegible]
```

Step 1：取得資料，看清結構（8 分鐘）

```
import pandas as pd

df = pd.read_csv("sales_sample.csv")

# [illegible]
print("=== [illegible] ===")
print(f"Shape: {df.shape}")
print(f"\n[illegible]")
print(df.dtypes)
```

在下方新增 Markdown cell，用你自己的話描述：「這份資料有幾筆記錄、幾個欄位，每個欄位代表什麼。」

Step 2：資料品質快速評估（10 分鐘）

```
# [illegible]
print("=== [illegible] ===")
print(df.isna().sum())
print()
print(df.isna().mean().map(lambda x: f"{x:.1%}").to_frame())

# [illegible]
print("\n=== [illegible] ===")
print(df.dtypes)

# [illegible]
print("\n=== [illegible] ===")
print(df.describe())
```

新增 Markdown cell，套用四維度框架，回答： - 完整性：哪些欄位有缺失？比例如何？ - 一致性：型態是否如預期？ - 準確性：有沒有不合理的數值（負金額、退貨率 > 100%）？ - 時效性：資料的日期範圍是什麼？

Step 3：找一個值得問的問題（9 分鐘）

```
# [illegible]
df.groupby("category")["amount"].sum().sort_values(ascending=False)
```

```
# [illegible]
df.groupby("region")["amount"].agg(["sum", "mean", "count"])
```

新增 Markdown cell，根據上面的輸出，寫下你的觀察： - 哪個發現讓你意外？ - 有什麼問題需要進一步確認？ - 如果要做一個完整分析，你的第一個分析假設是什麼？

Step 4：撰寫資料品質摘要（5 分鐘）

在 Notebook 最末新增一個 Markdown cell：

```
## [illegible]
- **[illegible]**：[illegible]
- **[illegible]**：[illegible]
- **[illegible]**：[illegible]
- **[illegible]**：[illegible]
- **[illegible]**：[illegible]
```

展示與討論（5 分鐘）

請 2-3 位學員分享他們的摘要，講師點評：「問題識別是否系統性？觀察是否和資料對應？」

工作坊 A 核心體驗：拿到一份新資料，不是立刻開始分析，而是先做系統性的健康檢查，把觀察記錄在 Notebook 裡。這個習慣，是資料分析師與「剛學會 pandas」的初學者之間的分水嶺。

S11 — 工作坊 B：資料品質健康檢查

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook 資料： *`dirty_sales.csv`*（課程頁面提供，故意包含各類資料問題）

目標： 套用 S04 和 S05 學到的四種污染類型與四維度評估框架，對一份含有已知問題的資料進行完整的品質稽核，並在 Notebook 中記錄每個問題和處理決策。

工作坊 B 任務說明

資料背景： `dirty_sales.csv` 是故意製造的「髒資料」，包含至少一個每種類型的問題。你的任務是找出所有問題，並說明你的處理決策。

Step 1：缺失值稽核（8 分鐘）

```
import pandas as pd
import numpy as np

df = pd.read_csv("dirty_sales.csv")

# ────
missing_report = pd.DataFrame({
    "count": df.isna().sum(),
    "pct": df.isna().mean() * 100
}).sort_values("pct", ascending=False)

print(missing_report)
```

新增 Markdown cell，對每個有缺失的欄位說明： - 缺失比例是多少？ - 這個欄位的缺失是否可以接受？ - 你的處理策略是什麼（刪除 / 填補 / 保留），理由是什麼？

Step 2：重複值稽核（7 分鐘）

```
# [redacted]
total_dups = df.duplicated().sum()
print(f"[redacted]{total_dups}")

# [redacted]
if total_dups > 0:
    display(df[df.duplicated(keep=False)].sort_values(df.columns.tolist()))

# [redacted]
key_col = 'order_id'
dup_keys = df[df.duplicated(subset=[key_col])][key_col].tolist()
print(f"\n[redacted] {key_col}[redacted]{dup_keys}")
```

Step 3：型態與離群值稽核（8 分鐘）

```
# [redacted]
print("[redacted]")
print(df.dtypes)

# [redacted]
try:
    pd.to_numeric(df["amount"], errors="raise")
    print("\namount [redacted]")
except:
    # [redacted]
    bad_rows = df[pd.to_numeric(df["amount"], errors="coerce").isna()]
    print(f"\namount [redacted] {len(bad_rows)} [redacted]")
    display(bad_rows.head())
```

```
# [redacted]
clean_amount = pd.to_numeric(df["amount"].astype(str).str.replace(r"[$,]", "", regex=True), errors
print(clean_amount.describe())

# [redacted]
q99 = clean_amount.quantile(0.99)
outliers = df[clean_amount > q99 * 3]
print(f"\n[redacted] > 3 [redacted]99[redacted]{len(outliers)} [redacted]")
```

Step 4：撰寫完整稽核報告（7 分鐘）

```
# [redacted]dirty_sales.csv
[
## [redacted]
- [redacted]
[
## [redacted]
- [redacted]
[
## [redacted] / [redacted]
```

```
[illegible]
[illegible]
### [illegible]
[illegible]
[illegible]
### [illegible]
[illegible]
[illegible]
[illegible]
[illegible]
```

工作坊 B 核心體驗：一份「稽核報告」不是程式碼的輸出，而是你的判斷和決策的記錄。別人看到你的 Notebook，應該能理解你為什麼做了這些清理，而不只是看到清理後的結果。

S12 — 模組總結：分析師的思維工具箱

核心訊息： M1 建立了資料分析師的兩件基本配備：資料思維框架（生命週期 + 品質評估 + 格式識別）和 Notebook 工作流（四合一設計 + 實戰技巧）。這兩件配備是後續所有分析技術的地基。

講師講解要點：

快速回顧 M1 學到的核心概念： - 資料分析生命週期的六個階段，以及每個階段的輸入輸出。 - 結構化資料的五個元素：列、欄、綱要、型態、鍵。 - 四種污染類型：缺失、重複、離群、型態錯誤，各有不同處理策略。 - 資料品質評估的四個維度：完整性、一致性、準確性、時效性。 - 常見資料格式：CSV、JSON、SQL、API，各自的結構特徵。 - Notebook 工作流的最佳實踐：cell 管理、Markdown 結構、執行順序控制。

現在能做的事： - 拿到一份新資料，能系統性地做初步健康檢查。 - 用 Notebook 把分析過程組織成有推論脈絡的敘述。 - 識別資料問題的四種類型，並提出初步處理策略。 - 讀懂 CSV / JSON / SQL / API 的基本結構。

還不能做的事（後續模組會教）： - 用 Python 實際執行清理和分析操作（M2 Python 分析基礎）。 - 用 NumPy 和 pandas 處理大規模表格資料（M3）。 - 用視覺化工具呈現分析結果（M4）。

M2 的問題預告：「你現在知道資料長什麼樣、知道哪裡有問題，但要用 Python 實際動手處理，還需要掌握基本的 Python 語法工具。M2 只教分析師真正需要的那些，不多教也不少教。」

資料思維是這門課最重要的能力之一 - 不是語法，而是「看到資料就知道該問什麼問題」的直覺。 - 這個直覺需要練習，工作坊 A 和 B 是起點，後續每個模組都有機會繼續訓練。

視覺建議： 課程地圖圖，高亮 M0 和 M1 為「已完成」（深色），M2 為「下一站」（邊框高亮），M3-M6 為「待解鎖」（淺灰）。在 M1 方塊內列出本模組涵蓋的核心概念關鍵字。M1 到 M2 的箭頭上標注：「資料思維 → Python 表達力」。

過渡語（模組結束語）：「帶著你在工作坊 B 寫的稽核報告去 M2，我們要學的是：如何用 Python 把你報告裡寫的那些清理步驟，真正執行出來。」

工作坊總結

工作坊	重點技能	預期輸出	核心體驗
工作坊 A	資料初探、品質快速評估、結構化記錄	一個有完整段落結構的探索 Notebook，包含資料品質摘要	拿到新資料的系統性工作流，不是從
工作坊 B	資料品質稽核、四種污染類型識別、清理決策	完整的資料稽核報告，稽核報告是判斷處理決策的記錄，不只是	

講師注意事項

1. S01 到 S03
是概念建立期，學員可能覺得抽象。建議多用真實資料截圖和具體例子，避免停留在純定義。
2. S02 生命週期 要強調「提問」是最重要的第一步，也是最常被跳過的。可以用一個壞問題 vs. 好問題的對比來讓概念具體化。
3. S04 四種污染
的「離群值不一定是錯誤」這個概念通常讓學員驚訝，花時間讓這個概念清楚，避免學員以後習慣性刪除所
4. S05 品質框架
要強調這個框架是可以重複套用的工具，讓學員把它記住或記在筆記裡，後面每次遇到新資料都拿出來用。

5. S06 資料格式 可以讓學員說說他們在工作或生活中遇過哪些格式，讓課程內容和現實連結。
6. S07 Excel vs. Python 通常引起討論，給 2-3 分鐘讓學員說說自己的 Excel
痛點，但注意不要讓討論偏向「說 Excel 壞話」，而是引導到「選對工具」的判斷力。
7. 工作坊 A 的重點是 Markdown
說明的品質，不是程式碼。如果學員只寫程式不寫觀察，主動引導。
8. 工作坊 B
的重點是決策記錄，不是找到所有問題。沒找到所有問題沒關係，但要能說清楚為什麼這樣處理找到的問題

課前準備清單（學員）

- [ ] 安裝 Anaconda 或 Miniconda（或已有 Python 3.10+ 環境）
- [ ] 確認可以執行 `jupyter notebook` 或 `jupyter lab`
- [ ] 下載範例資料 `sales_sample.csv` 和 `dirty_sales.csv`（課程頁面提供）
- [ ] 閱讀 M0 模組筆記（課程地圖與學習動機）

延伸閱讀

- McCallum, Q.E. (2012). Bad Data Handbook. O'Reilly Media. — 各種真實的資料品質問題案例
- Jupyter Notebook 官方文件：Markdown 語法與 cell 操作快捷鍵。jupyter.org
- Wickham, H. (2014). "Tidy Data." Journal of Statistical Software, 59(10). —
整潔資料的理論基礎。doi.org/10.18637/jss.v059.i10
- JSON 官方規格：json.org — 格式語法的完整定義

參考文獻與引用來源

產業數據

1. 「60-80% 時間花在資料清理」 — 原始來源：CrowdFlower (now Figure Eight). (2016). Data Science Report. 調查顯示資料科學家約 60% 時間用於清理與組織資料，19% 用於收集資料。此數據後被 Forbes、Harvard Business Review 等廣泛引用。另見 Kaggle. (2018). State of Data Science and Machine Learning Survey，報告清理時間約佔 15-25%。實際比例因產業與專案而異，本課程引用 60-80% 為常見範圍。

2. Jupyter Notebook 作為資料分析標準工具 — Project Jupyter. jupyter.org. JetBrains/PSF Python Developers Survey 2024 顯示 Jupyter 仍是資料科學工作者最常用的互動式環境之一。

3. Wickham, H. (2014). "Tidy Data." Journal of Statistical Software, 59(10), 1-23. — 「每一列是一筆觀測，每一欄是一個變數」的整潔資料原則。